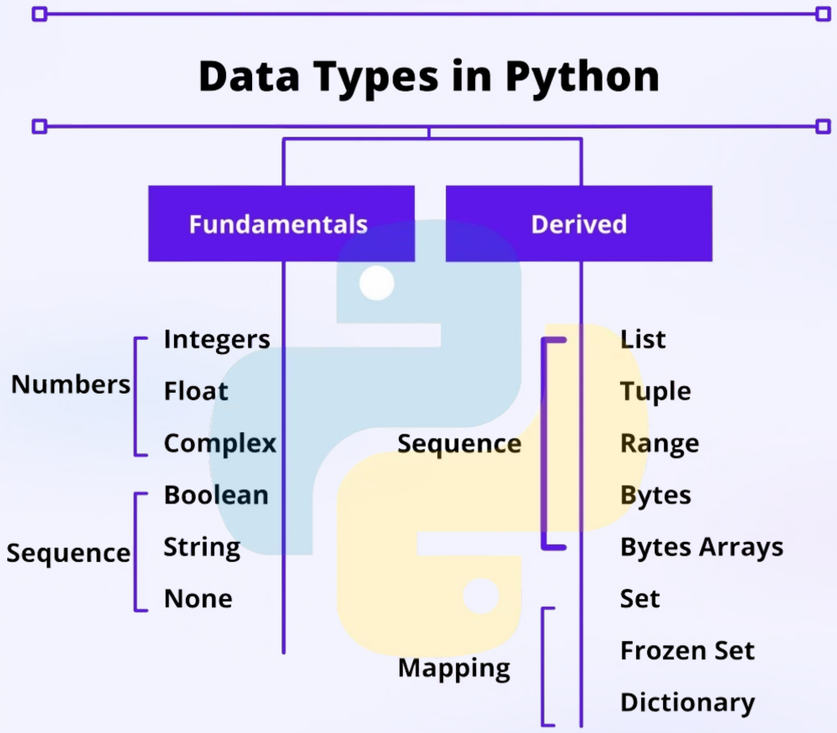
Hero VIRED | Virtual Education

Python Data Structures & Functions

Lists, Tuples, Sets, Dictionaries & Functions



www.herovired.com



Getting Started

Introduction to Data Structures

Data structures are fundamental to programming. They are specialised formats for organising and storing data, enabling efficient access and modification. Understanding them is crucial for writing clean, performant, and maintainable code.

Python offers several powerful built-in data structures: **Lists, Tuples, Sets, and Dictionaries**. We encounter these concepts daily in real-world scenarios, such as a shopping list (list), phone contacts (dictionary), or a unique collection of items (set).

Core Components

Lists in Python

Key Characteristics

- Ordered sequence of elements.
- Mutable (can be changed after creation).
- Allows duplicate values.

Enclosed in square brackets `[]`.

Practical Example

```
fruits = ["apple", "banana", "cherry"]
```

Here, `fruits` is a list. You can add or remove items, and the order is preserved.

Common Operations

`append()`: Adds an item to the end.

`remove()`: Deletes a specified item.

Slicing: Accesses a portion of the list (e.g., `fruits[0:2]`).

`sort()`: Sorts the list in place.

Core Components

Tuples in Python

Key Characteristics

- Ordered sequence of elements.

Immutable (cannot be changed after creation).

- Allows duplicate values.

Enclosed in parentheses `()`.

Practical Example

```
coordinates = (10, 20)
```

Once created, the values in `coordinates` cannot be altered. This makes tuples ideal for fixed data.

Use Cases

- Representing fixed geographical coordinates (e.g., latitude, longitude).
- Storing database records where immutability is desired.
- Function return values that should remain constant.

Core Components

Sets in Python

Key Characteristics

- Unordered collection of elements.

Unique values only (no duplicates).

- Mutable (can add/remove elements).

Enclosed in curly braces `{}`.

Practical Example

```
numbers = {1, 2, 3,
3}print(numbers) #
Output: {1, 2, 3}
```

Notice how the duplicate '3' is automatically removed, leaving only unique values.

Primary Uses

- Efficiently removing duplicate entries from a list.
- Fast membership testing (checking if an item exists in the set).
- Performing mathematical set operations (union, intersection, difference).

Dictionaries in Python

Dictionaries are powerful collections that store data as key-value pairs. Each key is unique and maps to a specific value, similar to a real-world dictionary where words (keys) have definitions (values).

Key Characteristics

- Unordered collection of items (Python 3.7+ preserves insertion order).
- Mutable (can add, remove, or modify key-value pairs).
- Keys must be unique and immutable (e.g., strings, numbers, tuples).
- Values can be of any data type and can be duplicated.

Practical Example

```
student = {"name": "John", "age": 20,
"course": "Computer Science"}
```

Access values using their corresponding keys:

```
print(student["name"]) # Output: John
```

Add a new key-value pair:

```
student["grade"] = "A"
```

Choosing the Right Tool

Comparing Data Structures

Data Structure	Order	Mutability	Duplicates
List	Ordered	Mutable	Allowed
Tuple	Ordered	Immutable	Allowed
Set	Unordered	Unique	Not Allowed
Dictionary	Key-Value Mapping	Mutable	Values Allowed, Keys Unique

Selecting the appropriate data structure for your task is crucial for efficient and effective programming. Consider the requirements for order, changeability, and uniqueness when making your choice.

Introduction to Functions

Functions are blocks of organised, reusable code that perform a single, related action. They allow you to break down complex problems into smaller, manageable chunks, making your code more readable, maintainable, and efficient.

Basic Syntax

```
def greet(name):
return "Hello " + name
```

`def`: Keyword to define a function.

`greet`: The name of the function.

`(name)`: Parameter(s) the function accepts.

`return`: Optional keyword to send a value back.

Key Benefits

Modularity: Divides code into logical sections.

Reusability: Avoids repetitive code; write once, use many times.

Readability: Easier to understand and debug.

- Abstraction: Hides complex implementation details.

Practical Function Examples

Simple Calculation

```
def add(a, b):    return a +
bresult = add(5, 3) # result is 8
```

This function takes two parameters and returns their sum, demonstrating input and output.

Real-World Scenario: Voting Eligibility

```
def check_eligibility(age):
if age >= 18:
   return "Eligible to vote"
 else:
return "Not eligible"
```

Functions can implement conditional logic, like this example checking for voting age.

Reusable Tool: Calculator Function

```
def calculate(operation, x, y):
if operation == "add":
return x + y
elif operation == "subtract":
return x - y# ... more operations
```

A single function can be designed to perform various tasks based on parameters, making it a versatile tool.

Wrap-Up & Practice Tasks

Recap

Lists: Ordered, mutable, allow duplicates.

Tuples: Ordered, immutable, allow duplicates.

Sets: Unordered, unique elements.

Dictionaries: Key-value pairs, mutable.

Functions: Reusable blocks of code for modularity.

Practice Tasks

Shopping Cart: Create a list named `shopping_cart`. Add items, remove items, and print the final cart.

Student Records: Use a dictionary to store details for a student (name, ID, grades). Access and update their information.

Factorial Function: Write a function `calculate_factorial(n)` that returns the factorial of `n`.

Unique Words: Given a list of words, write a function that returns a new list containing only the unique words.

Continue experimenting with these concepts to solidify your understanding. Happy coding!

Hero VIRED | Virtual Education

Thank You

www.herovired.com

Wait Nahi, *Great* Kar!